

SSH Brute-Force Detection — Lab Test Procedure

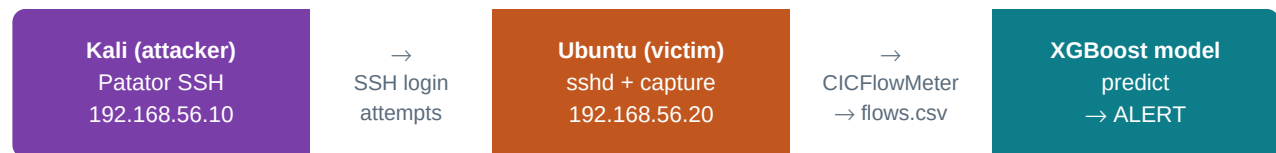
How to reproduce the CICIDS2018 SSH brute-force attack in an isolated two-VM lab and check whether the trained XGBoost model detects it. Attacker: Kali Linux (Patator). Victim: Ubuntu running an SSH server. All traffic stays on a private host-only network.

Authorized use only. Run this exclusively against virtual machines you own, on an isolated host-only network with no route to the internet or any third party. Brute-forcing a system you do not own is illegal. This procedure reproduces a published academic dataset's method to validate a defensive detector.

00 — HOW THE TEST WORKS

Attack → capture → flows → model → alert

The model does not read packets — it reads **flow rows** (one row per connection, with 80 statistics). So the victim captures the attack traffic and CICFlowMeter (the same tool that built CICIDS2018) converts it into rows, which the model then scores.



01 — WHAT YOU NEED

Host machine and two virtual machines

Machine	Role	Specs	Key software
Windows host	runs VirtualBox	12 GB RAM (close Chrome during test)	VirtualBox 7
Ubuntu VM	victim / SSH server	3 GB RAM, 2 CPU, 25 GB disk	openssh-server, CICFlowMeter, Python + xgboost
Kali VM	attacker	2 GB RAM, 2 CPU	Patator, rockyou wordlist

02 — NETWORK (do this first)

Put both VMs on the same private host-only network

In VirtualBox: **Tools** → **Network** → **Host-only Networks** → **Create** (gives vboxnet0, 192.168.56.x). For **each** VM → Settings → Network: Adapter 1 = **Host-Only** (vboxnet0) for the private link; Adapter 2 = **NAT** so the VM can download packages. After booting, note each VM's host-only IP with `ip a` (the 192.168.56.x address).

Verify from Kali before continuing: `ping 192.168.56.20` and `ssh testuser@192.168.56.20` should reach the Ubuntu VM.

03 — VICTIM: UBUNTU SETUP

SSH server, capture tools, and the model runtime

a. Install the SSH server, capture and build tools:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y openssh-server tcpdump default-jdk git \
    python3-pip python3-venv net-tools
sudo systemctl enable --now ssh
ip a # note the 192.168.56.x address
```

b. **Create the account the attack will target** (a weak password, so a normal login also works for the baseline test):

```
sudo adduser testuser # e.g. password: Password1
```

c. **Flow extractor — CICFlowMeter** turns captured packets into the 80-column rows the model was trained on. Two options:

- **CICFlowMeter (Java, matches the dataset best)** — download from the CIC / github.com/ahlashkari/CICFlowMeter, build with the included Gradle wrapper, and run it on a pcap. Needs default-jdk and libpcap.
- **Python cicflowmeter (easier)** — `pip install cicflowmeter`. Its column names differ slightly, so the detection script renames them to the model's 70 feature names.

d. **Model runtime** (Linux has no WMI, so no workaround is needed here):

```
python3 -m venv ~/detect && source ~/detect/bin/activate
pip install xgboost pandas numpy cicflowmeter
# copy from the Windows project into ~/detect/ :
# ssh_bruteforce_xgb.json model_features.json
```

04 — ATTACKER: KALI SETUP

Patator and a wordlist

Patator ships with Kali. Unpack the rockyou wordlist:

```
which patator || sudo apt install -y patator
sudo gunzip -k /usr/share/wordlists/rockyou.txt.gz # ignore if already unzipped
```

05 — RUN THE TEST

In order, so you get both a normal baseline and the attack

Step 1 — Start capture on the victim (Ubuntu)

Record all traffic on the host-only interface to a pcap (replace `enp0s3` with your interface from `ip a`):

```
sudo tcpdump -i enp0s3 -w ~/attack.pcap
```

Step 2 — Baseline: a normal SSH login (from Kali)

In a second Kali terminal, log in properly, run a couple of commands, then exit. These flows should later be classified Benign — the important check that the model is not just flagging all of port 22.

```
ssh testuser@192.168.56.20 # correct password, run 'ls', 'whoami', then 'exit'
```

Step 3 — Launch the SSH brute force (from Kali)

Point Patator at the victim only. Let it run about one to two minutes, then stop it with Ctrl+C.

```
patator ssh_login host=192.168.56.20 user=testuser \
password=FILE0 0=/usr/share/wordlists/rockyou.txt \
-x ignore:mesg='Authentication failed'
```

Step 4 — Stop the capture (Ubuntu)

Back in the tcpdump terminal, press Ctrl+C. You now have ~/attack.pcap containing the baseline login and the brute-force burst.

Step 5 — Convert packets to flow rows (Ubuntu)

Turn the pcap into a flows CSV with CICFlowMeter (Python version shown):

```
cicflowmeter -f ~/attack.pcap -c ~/flows.csv
```

Step 6 — Run the model to detect (Ubuntu)

The detection script loads the model, aligns the CSV columns to the 70 training features, predicts each flow, and prints an alert for every source IP with brute-force flows:

```
python detect_from_flows.py ~/flows.csv
```

06 — WHAT SUCCESS LOOKS LIKE

Expected output

```
[baseline] 192.168.56.10 -> :22 Benign (p=0.02)
[attack]   192.168.56.10 -> :22 SSH-Bruteforce (p=0.99)
[attack]   192.168.56.10 -> :22 SSH-Bruteforce (p=0.99) ...
>> ALERT: 192.168.56.10 made 214 SSH brute-force flows to port 22
```

Success = the normal login reads as Benign and the Patator burst is flagged as SSH-Bruteforce. If normal SSH is also flagged, the model over-relies on port 22 and would need retraining with more varied benign SSH — which is exactly what this test is for.

07 — THINGS TO EXPECT

Honest caveats

- **Column names must be aligned.** CICFlowMeter's CSV headers differ from the dataset's (e.g. "Total Fwd Packet" vs "Tot Fwd Pkts"). The detection script renames them to the model's 70 feature names; without this, prediction fails.
- **Real traffic is not CICIDS2018.** Different OS versions, timing and network mean the 100% test-set score will not carry over exactly. Measuring that gap is the point of the test.
- **Resources.** Windows + Kali + Ubuntu on 12 GB RAM is tight; close other apps. Offline (pcap then convert) is more reliable than live capture on a small VM.

The detection script `detect_from_flows.py` (with the CICFlowMeter→model column map) is provided separately in the project. Procedure compiled 2026-09-28.